

INDIA RUNS · DATA & AI CHALLENGE

Ranking candidates the way a great recruiter would.

An intelligent candidate-discovery system for the Redrob “Senior AI Engineer” role — that reads the **gap between what a profile says and what it means**, instead of counting keywords.

100,000 candidates → top 100

CPU-only · no network · ≤5 min

Pure Python · zero dependencies

Redrob Candidate Ranking System

Appr

THE TASK

Rank the best 100 of 100,000 — and prove you read the profiles

Input → Output

One job description + 100K structured candidate profiles → a ranked top-100 CSV with a score and a written justification for every row.

Scored on (hidden ground truth)

0.50·NDCG@10 + 0.30·NDCG@50 + 0.15·MAP + 0.05·P@10.
The top 10 is **half the score**.

Hard constraints

≤5 min wall-clock · ≤16 GB RAM · CPU-only
network OFF — no LLM/API calls during

And the dataset fights back

It is deliberately seeded with traps: **keyword stuffers** (off-domain people with perfect AI skill lists), **~80 honeypots** with impossible profiles (>10% in your disqualified), and **stale “perfect-on-paper”** candidates who aren’t actually reachable.

The challenge

This is a profile-reasoning problem, not a search problem

The organizers say it outright: “*the right answer is **not** the candidate with the most AI keywords — that’s a trap we built in.*” A single resume-vs-JD similarity score **amplifies** every trap.

What loses

- Cosine / embedding similarity → ranks the keyword stuffers up
- Skill-list matching → skills are pure noise (next slide)
- An LLM per candidate → forbidden, and won’t fit 5 min on CPU

What wins

- Read the **career history**: did they actually build ranking / retrieval, production?
- Encode the JD’s real rubric as transparent, auditable rules
- Weigh **availability** — unreachable ≠ hireable

We let the 100K profiles decide the architecture

Skills are noise

Every skill appears on ~12% of candidates, near-uniform across a huge vocabulary. A Marketing Manager is as likely to list "Kafka" as an engineer.

→ Never reward bare skill presence.

No hidden "Tier-5" gems

All 98,821 non-ML-titled candidates have zero retrieval/ranking evidence in their career text.

→ Embeddings add nothing. Dropped them — faster, simpler, trap-proof.

Honeypots are impossible

e.g. a skill used 92 months on a 3-year career; "expert" in 10 skills with 0 months; an 8-year career with 16 months of dates.

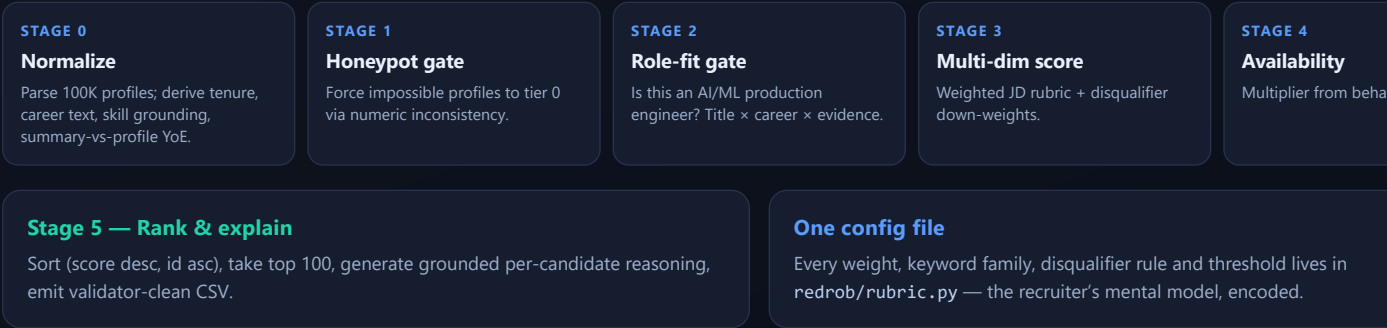
→ Catch them by numeric consistency, not guessing.

Where the real signal lives

Career **descriptions** separate the real from the fake: genuine ML engineers describe ranking, retrieval, recommendation and search work; stuffers list AI skill role descriptions are about sales, accounting, or design.

A five-stage, fully transparent pipeline

No LLM, no embeddings, no GPU in the ranking step — every score is explainable line by line.



Why the keyword stuffer collapses

Candidate: “Marketing Manager”

Headline: *“AI enthusiast | Building with LLMs.”* Skills: RAG, Embeddings, Fine-tuning LLMs, LangChain.

But every career stint is **sales, brand design, accounting** — and the summary admits “taking online courses on RAG.”

role_fit $\approx$ 0.05 → **ranked #27 of 68, score 0.058**

How the gate scores it

role_fit = title trajectory × career ML-ness × production-evidence in des

Skills never enter. An off-domain title with no ML career evidence cannot

by any skill list.

Guarded disqualifiers (so honest framing isn’t over-punished):

dabbler

pure-research

consulting-only

perception-only

title-c

big-tech-only

Explainable dimensions, then an availability reality-check

Stage 3 — weighted fit (JD rubric)

- role-fit (34%) · production-evidence (24%)
- experience band 5–9 yr (12%) · trajectory (10%)
- domain NLP/IR match (10%) · location (10%)

Each dimension is in [0,1] and reported per candidate — the ranking is auditable, not asserted.

Stage 4 — availability multiplier

The JD: *“perfect on paper but hasn’t logged in for 6 months with a 5% response rate, not actually available.”*

A multiplier (≈0.5–1.08) from response rate, recency, open-to-work, notice period, and GitHub activity. It **modulates** fit — it never manufactures it.

Effect: a Microsoft NLP engineer with a 7% response rate and an 8-month notice period drops out of the top 30.

RESULTS

The shortlist — and the numbers behind it

#	ROLE @ COMPANY	YRS	LOC	SIGNALS
1	Search Eng @ Sarvam AI	7.6	Gurgaon	resp 0.94 · active
2	RecSys Eng @ CRED	8.0	Noida	resp 0.77 · OTW
3	RecSys Eng @ Swiggy	6.0	Hyderabad	resp 0.91 · active
4	Staff MLE @ Paytm	7.0	Kochi	resp 0.95 · active
5	RecSys Eng @ Nykaa	7.7	Bangalore	resp 0.72 · active

Top-100 is 100% applied-ML/AI roles at product companies (AI/ML 29 · Fintech 8 · E-commerce 8 · ...). Zero stuffers.

0

honeypots in top 100

~65

full 100K run

1.8GB

peak RAM (limit 16)

0

external depen

PROOF, NOT PROMISE

Every trap is demonstrably handled

Honeypots All known impossible profiles forced to tier 0 (score -1.0). A self-test asserts 0% in the top 100 on every run — well under the 10% disqualification line.	Keyword stuffers The Marketing Manager with a perfect AI skill list lands at #27 / 0.058 — suppressed by the role-fit gate, not by a special case.	Stale & off-target Unreachable candidates down-weighted; tech careers nudged toward the cutoff, per preference for product/startup builders.
Built-in self-test Asserts: known honeypots flagged · known strong fits preserved · 0% honeypots in top 100 · score monotonic by rank · ties broken by candidate_id. Runs against pool.		

Honest, reproducible, and defensible

No fabricated anything

No ground truth ships with the data, so we report methodology and ablations — never a made-up NDCG. No fake confidence scores. No hallucinated skills in any reasoning string.

No LLM in the loop

The ranking step makes zero network/API calls and uses no GPU. An LLM was used only in *development* to analyze data and shape the rules — decisions are made honestly.

Trivial reproduction

Pure standard library — nothing to install, no version drift. One command regenerates the exact submission.

```
python rank.py --candidates candidates.jsonl --out submission.csv
```

Defensible by design

Every decision traces to a line in `rubric.py` and a fact in the data — ready to be explained through in the engineering interview.

SUMMARY

Reasoning, not keyword-matching — fast, transparent, trap-proof.

Deliverables

Runnable repo · top-100 CSV · this deck · live
sandbox demo.

Why it works

Encodes the JD's real rubric; reads careers, not
keywords; respects every compute rule.

What it proves

0 honeypots, 0 stuffers, all-applied-ML to
65 seconds on a CPU.